

We check for equality using the `==` operator ❸, which resolves to calling the `__eq__()` method of the `Rectangle` class. If the rectangles are the same size, we print out an appropriate message. If not, we check if the first rectangle is smaller than the second using the `<` operator ❹, which results in a call to the `__lt__()` method. If this comparison is also not `True`, we print the message for the first being larger than the second ❺. We didn't need to use the `>` operator in this program; however, since other client code might implement size comparisons differently, we've included the `__gt__()` method for completeness.

Finally, we draw all the rectangles in our list ❻.

Because we included the magic methods `__eq__()`, `__lt__()`, and `__gt__()` in the `Rectangle` class, we were able to use the standard comparison operators in a highly intuitive and readable way.

Here is the output from clicking on a number of different rectangles:

```
Clicked on first rectangle.  
Clicked on second rectangle.  
Rectangles are the same size.  
Clicked on first rectangle.  
Clicked on second rectangle.  
First rectangle is smaller than second rectangle.  
Clicked on first rectangle.  
Clicked on second rectangle.  
First rectangle is larger than second rectangle.
```

Math Operator Magic Methods

You can write additional magic methods to define what happens when client code uses the other arithmetic operators between objects instantiated from your class.

[Table 9-2](#) shows the methods that are called for the basic arithmetic operators.